Lab 1 — Common Issues

ECON 470 — Spring 2026

A handful of patterns came up across many of your submissions. None of them is fatal on its own, but together they account for most of the points the class lost. Read these alongside the answer key.

Problem 1, Q2 and Q6: a loss number is not an answer

The prompt asks for the *initial* and *final* loss values *and* their conversion to probabilities *and* a comparison to a baseline. Most of you reported the two loss numbers and stopped, or did the conversion but skipped the baseline.

The full answer at step 1 is

$$L \approx 3.36 \implies P = e^{-L} \approx 0.035 \approx \frac{1}{27},$$

and that last step is the whole point: the model at step 1 is essentially picking uniformly at random across 27 tokens (26 letters + BOS). Without the comparison to 1/27, you've reported a number; *with* it, you've explained what the number means. Q2 has the same structure for the final loss, where the comparison is to the initial loss (not to a baseline).

A common variant: reporting Q6's initial loss as ≈ 3.36 (the block_size=16 number) rather than re-running and recording the new run's ≈ 3.48. The point of Q6 is *the comparison between the two runs*, so the numbers have to come from the runs, not from memory.

Problem 1, Q4: the context-window question has *two* parts

Most of you got 23 characters and 45 names. The harder half is *what those numbers mean for training*. The model is trained with positional embeddings 0 through 15 only. For the 45 names longer than 16 characters, the model never sees the full name during a single forward pass — when it's predicting character 17, the embeddings for positions 0-15 are already "filled" and there's no learned positional embedding for position 16. The model is effectively blind to the start of the name when generating the end.

The numerical part is the easy half. The implication is what the question is testing.

Problem 2, Q7: the `Value` class is *not* a tokenizer

A few answers said the `Value` class "turns text into numbers" or "encodes the input." It does neither. The `Value` class is a wrapper around a single floating-point number that records *how that number was computed* — its parents in the computation graph, the operation that produced it, and the local derivative at that node. Tokenization happens elsewhere, before any `Value` exists.

The right framing: `Value` is the autodiff scaffolding. It's the thing that makes `loss.backward()` possible by remembering, for every intermediate quantity, which other `Value`s it came from and how. Without that bookkeeping, the model would still produce a forward-pass output, but you'd have no way to compute gradients.

Problem 2, Q8: `backward()` does *not* update parameters

This was the single most common conceptual error in P2. Several answers wrote that `L.backward()` "modifies the parameters" or "trains the model." It does neither.

`L.backward()` computes the gradient $\partial L/\partial w$ for every `Value` in the graph that contributed to L, and stores those gradients in each `Value`'s `.grad` attribute. The parameters themselves are unchanged after `backward()` returns. The *next* step in the training loop — `optimizer.step()`, which is Adam — reads those gradients and adjusts each parameter accordingly.

The cleanest mental model is two-stage: `backward()` *fills out the .grad attributes*; the optimizer *uses them to step*. The training loop pattern

```
loss.backward()
optimizer.step()
optimizer.zero_grad()
```

makes the separation explicit. Each line does exactly one thing.

Problem 2, Q10: show the chain rule

A few of you reported $MP_L = 0.625$ and $MP_K = 5/12$ correctly but with no derivation visible — just the final numbers. The question asks for the chain rule. The clean version is one line of substitution:

$$\frac{\partial Y}{\partial L} = \frac{1}{\rho} Z^{1/\rho - 1} \cdot \alpha \rho L^{\rho - 1} = \alpha L^{\rho - 1} Z^{1/\rho - 1}$$

with $\rho = 0.5$, $\alpha = 0.5$, $L = 4$, $Z = 2.5$:

$$MP_L = 0.5 \cdot 4^{-0.5} \cdot 2.5 = 0.5 \cdot 0.5 \cdot 2.5 = 0.625.$$

The point of Q10 is that you can *see* what the autodiff system is doing under the hood when you call `Y.backward()`. The numerical answer alone doesn't demonstrate that — the chain rule does.

Problem 3, Q14: "different" doesn't mean "rephrased"

Q13 asks for three things MicroGPT and MicroOLS share. Q14 asks for three things they *differ on*. A few answers gave the same three items twice — once as "shared structure" and once as "different but in the same shared structure." That's not what the question is asking.

The clean differences are at the level of:

1. **Model.** Linear in three regressors vs. transformer with embeddings, attention, MLP.
2. **Loss function.** Mean squared error vs. cross-entropy.
3. **Inference.** Sampling from $\mathcal{N}(\hat{y}, s_e^2)$ vs. sampling from a categorical over 27 tokens.

If your three "differences" can be summarized as "they have different code," you haven't named differences. Differences are about *what kind of object the model is*, *what it predicts*, and *how its output distribution is shaped* — three places where the architectural choices diverge.

Problem 3, Q17: the range comes from s_e, not from luck

Most of you reported the range of 20 sampled sales values correctly (something like 2,000-28,000 units). The question's substance — *why* the range is that wide — depends on the residual standard error. From the OLS fit, $s_e \approx 0.79$ in log-space. A $\pm 2\sigma$ band is therefore ± 1.58 in $\ln Q$, which exponentiates to a multiplicative factor of $e^{1.58} \approx 4.9$. So a 2σ window in log-sales spans nearly a $5\times$ multiplier in raw units — and a 4σ window (which 20 draws will populate fairly densely) spans a $\sim 25\times$ multiplier.

Without that calculation, "the range is large" is just a description. With it, you've identified the *source* of the variation, which is the residual noise term in the regression — the same role that $\varepsilon \sim N(0, s_e^2)$ plays in any OLS-based forecast.

Problem 3, Q19: cite Gans

The prompt asks you to "use the Gans framework of *prediction* and *judgment*" to compare MicroGPT and MicroOLS. Several answers engaged with the prediction-vs-judgment distinction in spirit but never named Gans, never quoted his framing, and never connected the lab to Chapter 1 of the book.

The strongest answers had three moves:

1. **What's prediction in each system?** Identify the conditional distribution being learned (next character given prefix; sales given price and brand).
2. **What's the shared substrate?** `Value` + `backward()` + Adam — same engine, different loss and architecture.
3. **What's the judgment?** The choices the engine cannot make for you: what to predict, what loss function to use, what architecture to commit to, what to *do* with the prediction.

Connecting that third bullet back to Gans's "prediction is cheap, judgment is dear" line is what the question is asking for. A reasonable answer without Gans was worth 7-8/10; a complete answer with Gans was worth 10/10.

A note on showing work

Two patterns made grading harder than it needed to be:

1. **Hand calculations only in figures.** Several writeups had Q9-Q11 as photographed handwritten work pasted into the PDF. That's fine for a graph or a sketched derivation, but for a number you also report in plain text, it doubles the grader's work. If you photograph your math, also transcribe the final number into the body of the writeup.
2. **Skipping problem numbers.** A few writeups jumped from Q6 directly to Q9 with no Q7/Q8 in between, or merged two problems under one header. Number every answer; it's the simplest way to signal "I addressed this."

The cleanest submissions had every numbered question, every comparison the prompt asked for, and one or two calculations explicitly tied to a baseline (the random-token baseline in Q2, the residual standard error in Q17, the closed-form OLS in Q16). That structure is what turns a writeup from "answers" into "an analysis."